Fondamenti di Calcolo Parallelo

Calcolo Parallelo e Distribuito, Prof.ssa L. Marcellino · Dispensa di ripasso, Fase A · v2 con derivazioni e citazioni

Fonti: **Lez1** (motivazioni), **Lez6** (parametri di valutazione), **Lez7** (legge di Ware-Amdahl), **Formulario Complessità**. Notazione delle dispense: S_p, E_p, O_h (all'orale anche S(p), E(p)).

1. Motivazioni del calcolo parallelo
2. Parametri di valutazione: T_1, T_p, S_p, O_h, E_p
3. Legge di Ware-Amdahl
4. Discrepanze con le slide (verifiche numeriche)
5. Come esporlo all'orale

Legenda: Lez6 s.42 = passaggio preso da Lezione 6, slide 42 · derivazione mia = passaggio non presente nelle slide, ricavato da me · ≠ slide = risultato che non coincide con la slide (verifica in §4).

---

1. Motivazioni del calcolo parallelo

INTUIZIONE

Molti problemi su larga scala sono caratterizzati dalla necessità di una **soluzione in tempo reale (o in tempo utile)** Lez1 s.9. Lo scopo di un supercalcolatore è *risolvere problemi facendogli fare i calcoli al nostro posto, più velocemente possibile* Lez1 s.10.

Problema	Vincolo su τ	Fonte
Query a un motore di ricerca	$\tau < 1$ s	Lez1 s.21–22
Intercettazione di un missile (Patriot vs Scud)	$\tau < 2$ min dal lancio	Lez1 s.24–25
Previsione meteo a 2 giorni	$\tau \leq 30$ min	Lez1 s.27–28

DERIVAZIONE: POTENZA RICHIESTA DALLA PREVISIONE METEO

1. **Ipotesi sul dominio** Lez1 s.26: superficie $20 \cdot 10^6$ km², quota 20 km, quindi volume $(20 \cdot 10^6) \cdot 20 = 4 \cdot 10^8$ km³. Si scrive un'equazione per ogni punto della griglia, che coinvolge almeno i punti vicini.
2. **Costo totale** Lez1 s.27: circa $2 \cdot 10^{17}$ flop per prevedere i prossimi 2 giorni.
3. **Tempo su un laptop** da 10^{11} flop/s Lez1 s.27: $\tau = 2 \cdot 10^{17} / 10^{11} = 2 \cdot 10^6$ s. Convertito: $2 \cdot 10^6 / 86400 \approx 23.1$ giorni (la slide arrotonda a "24 giorni").
4. **Vincolo** Lez1 s.28: $\tau \leq 30$ min $= 1.8 \cdot 10^3$ s.
5. **Potenza minima**: $R \geq 2 \cdot 10^{17} / 1.8 \cdot 10^3 \approx 1.11 \cdot 10^{14}$ flop/s ≈ 111 Tflop/s derivazione mia ≠ slide. La slide scrive "$1 \cdot 10^{14}$ flop/s, > 900 Tflop/s" e conclude *circa 1 Pflop/s* Lez1 s.28; vedi §4.

In sintesi, aumentare le prestazioni consente di **(i)** risolvere problemi in tempo reale (utile) e **(ii)** risolvere problemi di grandi dimensioni Lez1 s.29.

1.1 Modello del tempo di esecuzione

DEFINIZIONE FORMALE

Lez1 s.32

$$\tau = k \cdot T(N) \cdot \mu$$

- **T(N)**: complessità computazionale dell'algoritmo; *dipende dal metodo risolutivo scelto*.
- **μ**: tempo di esecuzione di un'operazione floating-point; *dipende dall'ambiente di calcolo*.
- **k**: costante moltiplicativa aleatoria (tempi di I/O, ritardi di scrittura in memoria).

Leva	Come si agisce	Limite	Fonte
↓ μ	tecnologia (miniaturizzazione)	**limiti tecnologici**: surriscaldamento, distanza d CPU–memoria	Lez1 s.33
↓ T(N)	ottimizzando l'algoritmo	**limiti teorici**: algoritmi ottimali (sistema diagonale $x_i = y_i/a_i$, T(n) = n)	Lez1 s.34
↓ k	quantum computing	ancora in fase **embrionale**	Lez1 s.35
T(N) → T_{np}(N)	**calcolo parallelo con np unità processanti**	oggetto del corso	Lez1 s.36

DEFINIZIONE DI CALCOLO PARALLELO

Lez1 s.37

Impiego simultaneo di risorse di calcolo multiple per risolvere un unico problema, spezzandolo in parti discrete elaborabili contemporaneamente, ovvero che possono essere eseguite *in modo seriale su differenti CPU*.

Misura delle prestazioni. Le prestazioni sono **application-dependent** Lez1 s.10. La TOP500, pubblicata due volte l'anno dal 1993, usa R_{max}: la performance massima ottenuta con HPL, la versione parallela del benchmark LINPACK (Dongarra), che risolve un sistema lineare denso Ax = b Lez1 s.11, 14–15.

TRADE-OFF: "FERRARI O PANDA? … DIPENDE DALL'USO"

Lez1 s.19–20

- **Pro:** per problemi su larga scala o con vincoli di tempo reale, il parallelismo è la via percorribile quando μ e T(N) sono al limite.
- **Contro:** decomporre il problema introduce lavoro aggiuntivo (ad es. la collezione dei risultati). Questo lavoro si formalizza come **overhead** (§2.4) e su problemi piccoli può annullare il guadagno.
- Non tutti i problemi sono completamente parallelizzabili Lez6 s.28, e da qui nasce la legge di Ware-Amdahl (§3).

2. Parametri di valutazione di un algoritmo parallelo

Avendo strategie e strumenti, la domanda è: *vale la pena implementare la strategia di parallelizzazione?* Per rispondere bisogna **valutare l'efficienza di un algoritmo in ambiente di calcolo parallelo** Lez6 s.4–5 .

2.1 $T_1(N)$ e $T_p(N)$

INTUIZIONE

In un algoritmo sequenziale il numero complessivo di operazioni è uguale al numero dei passi temporali Lez6 s.10 . In un algoritmo parallelo invece **non è uguale** Lez6 s.13 : il calcolatore parallelo esegue più operazioni *concorrentemente*, cioè allo stesso passo temporale. Il tempo di esecuzione non è quindi proporzionale alla complessità di tempo, e **la complessità di tempo non è adatta a misurare l'efficienza di un algoritmo parallelo** Lez6 s.14 .

DEFINIZIONE FORMALE

- **$T_1(N)$** = complessità computazionale su 1 processore; $\tau_1 = T_1(N) \cdot k \cdot \mu$ Lez6 s.7 Lez7 s.3 .
- **$T_p(N)$** = complessità computazionale su p processori, misurata in **passi temporali** ($t_{calc} = \mu$, tempo di un'addizione); $\tau_p = T_p(N) \cdot k \cdot \mu$ Lez6 s.21 Lez7 s.3 .

DERIVAZIONE 1: SOMMA DI DUE VETTORI, $T_p(N) = N/P$

1. **Sequenziale**: $c_i := a_i + b_i$ per $i = 0,\dots,N-1$, eseguendo le N addizioni una per volta Lez6 s.8 . Quindi $T_1(N) = N$, ad es. $T_1(16) = 16$ Lez6 s.9, 18 .
2. **Schema di calcolo** (decomposizione del dominio): il vettore è diviso in p blocchi contigui di N/p componenti. Con $p = 2$: $i = 0,\dots,7$ e $i = 8,\dots,15$; con $p = 4$: blocchi da 4 Lez6 s.11–12 . Ogni core esegue le sue N/p addizioni e i blocchi sono indipendenti, quindi **non serve altro** Lez7 s.14 : non c'è una fase di raccolta.
3. **Passi temporali**: tutti i core lavorano allo stesso passo, quindi i passi sono quelli di un singolo core: $T_2(16) = 8$, $T_4(16) = 4$, $T_8(16) = 2$ Lez6 s.19–23 .
4. **Generalizzazione** Lez6 s.23 Formulario p.2 :

$$T_1(N) = N \qquad T_p(N) = \frac{N}{p}$$

La slide ottiene la formula generalizzando la tabella; l'argomento "un core, N/p addizioni" del passo 2 è derivazione mia . Si suppone p divisore di N.

DERIVAZIONE 2: SOMMA DI N NUMERI, $T_1(N) = N - 1$

1. Algoritmo sequenziale: $s := a_0$; poi $s := s + a_i$ per $i = 1,\dots,N-1$. Le addizioni sono $N - 1$, una per passo: $T_1(16) = 15$ Lez6 s.29, 33 .
2. Quindi $T_1(N) = N - 1$ Formulario p.1 .

DERIVAZIONE 3: SOMMA DI N NUMERI, I STRATEGIA

1. **Fase locale** (decomposizione del dominio): ogni core somma i suoi N/p numeri in $N/p - 1$ addizioni; i p core lavorano in parallelo, quindi la fase costa $N/p - 1$ passi. Con $p = 2$, $N = 16$: 7 passi, 14 addizioni (7 per core) Lez6 s.30 .
2. **Collezione dei risultati**: un solo core somma le p somme parziali, una alla volta, quindi $p - 1$ addizioni sequenziali. Con $p = 2$: 1 addizione al passo 8 Lez6 s.30 .

3. **Totale**:

$$T_p(N) = \left(\frac{N}{p} - 1\right) + (p - 1)$$

Formulario p.1. Le slide mostrano solo il caso p = 2; la generalizzazione a p qualsiasi è derivazione mia. Verifica: N = 16, p = 2 dà 7 + 1 = 8 ✓ Lez6 s.34.

DERIVAZIONE 4: SOMMA DI N NUMERI, II STRATEGIA (RACCOLTA AD ALBERO)

1. **Fase locale**: come nella I strategia, N/p − 1 passi. Con p = 4, N = 16: passi 1–3, 12 addizioni Lez6 s.31, 35.
2. **Collezione ad albero**: a ogni passo i core si accoppiano e uno dei due somma la somma parziale dell'altro alla propria, quindi il numero di somme parziali si dimezza: p → p/2 → … → 1. Con p = 4: 2 addizioni al passo 4 e 1 al passo 5 Lez6 s.31. Con p = 8: passi 2, 3, 4 Lez6 s.36.
3. **Numero di passi della raccolta**: servono k passi con $p/2^k = 1$, cioè $k = \log_2 p$ (p potenza di 2) derivazione mia.
4. **Totale** Lez6 s.38, dove la slide scompone $T_2 = 7+1$, $T_4 = 3+2$, $T_8 = 1+3$:

$$T_p(N) = \frac{N}{p} - 1 + \log_2 p$$

QUALE ALGORITMO USARE PER T_1?

Lez7 s.7–10

- **Prima scelta:** T_1 = complessità dell'algoritmo parallelo su 1 unità processante. S_p dice quanto l'algoritmo *si presta all'implementazione* su un'architettura parallela. Svantaggio: potrebbe eseguire più operazioni del necessario Lez7 s.8.
- **Seconda scelta:** T_1 = complessità del miglior algoritmo sequenziale. S_p misura la riduzione *effettiva* della complessità. Difficoltà: individuare il "miglior" algoritmo sequenziale Lez7 s.9.
- **Convenzione del corso: prima scelta** Lez7 s.10.

2.2 Speed-up

INTUIZIONE

Con p core ci aspettiamo che T_1 sia p volte T_p, cioè di ridurre di p volte il tempo di esecuzione Lez6 s.16.

DEFINIZIONE FORMALE

Lez6 s.26, 41 Lez7 s.4

$$S_p = \frac{T_1}{T_p} \qquad S_p^{\text{ideale}} = p$$

Lo speed-up misura la **riduzione del tempo di esecuzione** rispetto all'algoritmo su 1 core. La slide scrive $S_p < p$; la forma corretta in generale è $S_p \leq p$ (§4).

DERIVAZIONE 5: SPEED-UP DEI DUE NUCLEI

1. **Full parallel** (somma di vettori): $S_p = N / (N/p) = p$, quindi speed-up ideale. Le slide lo enunciano senza fare il passaggio Lez6 s.27: *gli algoritmi full parallel hanno tutti speedup ideale*.
2. **Somma di N numeri, II strategia**:

$$S_p = \frac{N - 1}{\frac{N}{p} - 1 + \log_2 p}$$

forma chiusa mia. Numericamente, con N = 16: $S_2 = 15/8 = 1.875$, $S_4 = 15/5 = 3.00$, $S_8 = 15/4 = 3.75$ Lez6 s.40.

3. p = 8 è il più veloce, ma *ho utilizzato 8 processori per avere un incremento di quasi 4 volte* Lez6 s.46–47. Quindi **lo speed-up da solo non basta** a valutare l'efficienza Lez6 s.48.

2.3 Efficienza

INTUIZIONE

Usare più processori **non garantisce** algoritmi paralleli "efficaci", cioè che sfruttano tutte le risorse della macchina Lez6 s.50. Bisogna rapportare lo speed-up al numero di processori Lez6 s.49.

DEFINIZIONE FORMALE

Lez6 s.52 Lez7 s.6

$$E_p = \frac{S_p}{p} \qquad E_p^{\text{ideale}} = \frac{S_p^{\text{ideale}}}{p} = 1$$

L'efficienza **misura quanto l'algoritmo sfrutta il parallelismo del calcolatore**.

DERIVAZIONE 6: FORME EQUIVALENTI DELL'EFFICIENZA

derivazione mia

1. Sostituendo $S_p = T_1/T_p$: $E_p = \frac{T_1}{pT_p}$, cioè il rapporto tra lavoro utile (T_1) e lavoro pagato (p core per T_p passi).
2. Poiché $T_p \geq T_1/p$ (p core non possono fare più di p operazioni per passo), vale $0 < E_p \leq 1$.
3. Somma di N numeri, II strategia: $E_p = \frac{N-1}{N-p+p\log_2 p}$. Numericamente, con N = 16: $E_2 = 15/16 \approx 0.94$, $E_4 = 15/20 = 0.75$, $E_8 = 15/32 \approx 0.47$ ✓ Lez6 s.49.

2.4 Overhead totale

INTUIZIONE

pT_p è il lavoro complessivo pagato e T_1 il lavoro utile. La differenza misura **quanto NON si è riusciti a parallelizzare** Lez7 s.5.

DERIVAZIONE 7: DEFINIZIONE E FORMULA GENERALE DELLO SPEED-UP

1. **Definizione** Lez6 s.42 Lez7 s.5: $O_h = pT_p - T_1$
2. **Si ricava T_p** Lez6 s.42: $T_p = (O_h + T_1)\ /\ p$
3. **Si sostituisce in S_p** e si divide numeratore e denominatore per T_1 Lez6 s.42:

$$S_p = \frac{T_1}{(O_h + T_1)\ /\ p} = \frac{pT_1}{O_h + T_1} = \frac{p}{\frac{O_h}{T_1} + 1}$$

4. **Conseguenze** derivazione mia: $S_p = p \Leftrightarrow O_h = 0$, ed $E_p = 1/(O_h/T_1 + 1)$. Il degrado è governato dal **rapporto O_h/T_1**: è l'idea alla base dell'isoefficienza (Lez10).

DERIVAZIONE 8: OVERHEAD DELLA SOMMA DI N NUMERI

II strategia Lez6 s.43:

$$O_h = p\left(\frac{N}{p} - 1 + \log_2 p\right) - (N-1)$$

$$= N - p + p\log_2 p - N + 1 = p\log_2 p - p + 1$$

La slide scrive correttamente la riga intermedia "(N − p + p $\log_2$p − N + 1)", ma nel risultato incorniciato perde il +1 ("= p $\log_2$p − p") ≠ slide; verifica in §4.

I strategia derivazione mia, partendo da Formulario p.1:

$$O_h = p\left[\left(\frac{N}{p} - 1\right) + (p-1)\right] - (N-1) = N - p + p^2 - p - N + 1 = (p-1)^2$$

p	O_h II str. corretto	verifica $pT_p - T_1$ (N = 16)	valore della slide (Lez6 s.43)	O_h I str. $(p-1)^2$	verifica I str. (N = 16)
2	1	2·8 − 15 = 1	**2**	1	2·8 − 15 = 1
4	5	4·5 − 15 = 5	**8**	9	4·6 − 15 = 9
8	17	8·4 − 15 = 17	**24**	49	8·8 − 15 = 49

In entrambe le strategie O_h non dipende da N e cresce con p: *al crescere di p l'overhead aumenta* Lez6 s.43. La II strategia cresce come p log p, la I come p^2, quindi la II è preferibile per p grande osservazione mia.

TRADE-OFF: SPEED-UP VS EFFICIENZA

Per N = 16, **p = 8 minimizza il tempo** e *è da preferire se si vuole il minor tempo possibile* Lez6 s.44. Invece **p = 2 massimizza lo sfruttamento dei processori** Lez6 s.49. La scelta dipende dall'obiettivo: tempo di risposta (vincolo di tempo reale) oppure uso efficiente delle risorse.

2.5 Riepilogo dal Formulario

Algoritmo	$T_1(N)$	$T_p(N)$, I strategia	$T_p(N)$, II strategia	Fonte
Somma di N numeri	N − 1	(N/p − 1) + (p − 1)	(N/p − 1) + $\log_2$p	Formulario p.1
Somma di due vettori	N	N/p (full parallel)		Formulario p.2
Prodotto scalare	2N − 1	(2N/p − 1) + (p − 1)	(2N/p − 1) + $\log_2$p	Formulario p.2

3. Legge di Ware-Amdahl

INTUIZIONE

Il punto di partenza per scrivere un buon algoritmo parallelo è aver scritto un ottimo algoritmo sequenziale Lez7 s.12 . Ogni algoritmo contiene operazioni che **devono** essere eseguite in sequenziale. Quelle non traggono vantaggio dai core aggiuntivi e pongono un limite allo speed-up.

DERIVAZIONE 9: DALLA DECOMPOSIZIONE ALLA LEGGE

1. **Ipotesi: decomposizione di T_1** Lez7 s.13 : $T_1 = T_s + T_c$, dove T_s sono le operazioni che devono essere eseguite esclusivamente in sequenziale e T_c quelle che *potrebbero* essere eseguite concorrentemente.
2. **Ipotesi: la parte concorrente si divide perfettamente tra i p core** Lez7 s.16 :
$$T_p = T_s + \frac{T_c}{p}$$
Esempi con p = 2 Lez7 s.14–18 : somma di vettori, n = 4: $T_s = 0$, $T_c = 2+2$, quindi $T_2 = 0 + 4/2 = 2$. Somma di n = 4 numeri: $T_s = 1$ (l'addizione finale), $T_c = 2$, quindi $T_2 = 1 + 2/2 = 2$.
3. **Normalizzazione** Lez7 s.20 : $T_1 = 1$, $T_s = \alpha$, $T_c = 1 - \alpha$, da cui $T_p = \alpha + (1 - \alpha)/p$.
4. **Sostituzione in S_p** Lez7 s.20 :
$$S_p = \frac{T_1}{T_p} = \frac{1}{\alpha + \frac{1-\alpha}{p}} \qquad \text{(Legge di Ware-Amdahl)}$$
5. **Legame con l'overhead** derivazione mia : con $T_1 = 1$, $O_h = pT_p - T_1 = p\alpha + (1 - \alpha) - 1 = \alpha(p - 1)$. Sostituendo nella formula generale Lez7 s.20 , che dice *"ricordiamocelo questo"*: $S_p = p/(\alpha(p-1) + 1)$. Moltiplicando per p numeratore e denominatore della legge si ottiene la stessa espressione ✓.

DERIVAZIONE 10: ANALISI ASINTOTICA

1. **Full parallel** Lez7 s.21 : $\alpha = 0$, quindi $S_p = 1/(1/p) = p \to \infty$ per $p \to \infty$.
2. **Con collezione dei risultati** Lez7 s.21 : $\alpha > 0$. Per $p \to \infty$ il termine $(1-\alpha)/p \to 0$, quindi
$$\lim_{p \to \infty} S_p = \frac{1}{\alpha}$$
Lo speed-up è limitato superiormente indipendentemente da p.
3. **p fissato, $\alpha \to 0$** Lez7 s.29–30 : $\alpha \to 0$ e $1 - \alpha \to 1$, quindi $S_p \to 1/(1/p) = p$ ed $E_p \to 1$.

DERIVAZIONE 11: CHI È A NELLA SOMMA DI N NUMERI (P = 2)

derivazione mia

1. Con p = 2 l'unica addizione sequenziale è quella finale Lez7 s.15, 18 : $T_s = 1$, $T_1 = n - 1$, quindi $\alpha = 1/(n - 1)$.
2. Valori: $n = 8 \to 1/7 \approx 0.14$; $n = 16 \to 1/15 \approx 0.067$; $n = 32 \to 1/31 \approx 0.032$; $n = 64 \to 1/63 \approx 0.016$ (le colonne α di Lez7 s.22, 26 riportano gli stessi valori, arrotondati).
3. Esempio di calcolo, n = 8: $S_2 = 1/(1/7 + (6/7)/2) = 1/(4/7) = 1.75$ ✓ Lez7 s.26 . Coincide con $T_1/T_2 = 7/4$.

3.1 Esempi numerici delle slide, ricalcolati

n = 32 fissato, p variabile Lez7 s.22–23				
p	T_p (II str.)	$S_p = 31/T_p$	E_p esatta	E_p slide
2	15+1 = 16	1.94	0.97	0.95
4	7+2 = 9	3.44	0.86	0.85
8	3+3 = 6	5.17	0.65	0.6
16	1+4 = 5	6.20	0.39	**0.3**

Gli S_p della slide (1.9, 3.4, 5.1, 6.2) coincidono con T_1/T_p. Speed-up ed efficienza **degradano** Lez7 s.23.

p = 2 fissato, n variabile Lez7 s.26–28				
n	$\alpha = 1/(n-1)$	$S_2 = (n-1)/(n/2)$	E_2 esatta	slide S / E
8	0.143	1.75	0.875	1.75 / 0.875
16	0.067	1.875	0.94	1.8 / 0.9
32	0.032	1.94	0.97	1.9 / 0.96
64	0.016	1.97	0.98	1.96 / 0.99

L'efficienza **tende all'efficienza ideale** Lez7 s.28. Le differenze con la slide sono solo arrotondamenti.

OSSERVAZIONE: α NON È COSTANTE IN P

derivazione mia La slide etichetta $\alpha = 0.032$ come costante al variare di p Lez7 s.22. Applicando davvero Ware-Amdahl con α fisso si otterrebbe $S_{16} = 1/(0.032 + 0.968/16) \approx 10.8$, non 6.2. Nella somma con II strategia la parte non parallelizzabile sono i $\log_2 p$ passi di raccolta, che **crescono con p**: il degrado reale è quindi più rapido di quello previsto con α fisso. Anche la colonna "$(1-\alpha)/p$" della slide (0.968, 0.903, …) non coincide con $(1 - 0.032)/p$ (0.484, 0.242, …).

DERIVAZIONE 12: IL P_{MAX} DELLA SOMMA DI N NUMERI (II STRATEGIA)

derivazione mia

1. A N fissato, si cerca il p che minimizza $T_p = N/p - 1 + \log_2 p$ (trattando p come variabile continua).
2. Si deriva: $dT_p/dp = -N/p^2 + 1/(p \ln 2) = 0$, quindi $p^* = N \ln 2 \approx 0.69\ N$.
3. Verifica con N = 16: $p^* \approx 11$. Tra le potenze di 2, $T_8 = 1 + 3 = 4$ e $T_{16} = 0 + 4 = 4$, cioè oltre p = 8 il tempo non migliora più, mentre l'efficienza continua a scendere. È il p_{max} di Lez7 s.34.

TRADE-OFF E CONCLUSIONI

Lez7 s.24, 31–34

1. **n fissato, p crescente:** esiste $\mathbf{p_{max}}$, il miglior numero di processori per l'algoritmo in esame; superato questo valore *le prestazioni peggiorano*: non conviene usare un elevato numero di core.
2. **p fissato, n crescente:** $\alpha \to 0$ ed $E_p \to 1$, ma esiste $\mathbf{n_{max}}$, la massima dimensione che la macchina può memorizzare o elaborare, perché le risorse hardware sono limitate.
3. **Strategia:** *fissare il miglior numero p di CPU e aumentare la dimensione del problema*. Il legame quantitativo tra n e p a efficienza costante è l'**isoefficienza** (Lez10).

4. Discrepanze con le slide (verifiche numeriche)

Dove	La slide dice	Valore corretto	Verifica numerica
Lez6 s.43	$O_h = p \log_2 p - p$; tabella 2, 8, 24	**$O_h = p \log_2 p - p + 1$**; 1, 5, 17	$N = 16$, $p = 8$: $T_8 = 4$, $pT_p - T_1 = 32 - 15 = 17$. Anche $S_8 = 8/(17/15 + 1) = 3.75$ ✓ Lez6 s.40. Con 24 si avrebbe $8/(24/15 + 1) \approx 3.08$ ✗
Lez6 s.52, Lez7 s.6	"si definisce il rapporto E_p su p"	rapporto **S_p su p** (refuso)	la formula sotto il testo è $E_p = S_p/p$
Lez6 s.26, 41; Lez7 s.4	$S_p < p$	**$S_p \leq p$**	somma di vettori: $S_p = N/(N/p) = p$, uguaglianza Lez6 s.27
Lez1 s.28	$1 \cdot 10^{14}$ flop/s, "> 900 Tflop/s"	$\approx 1.11 \cdot 10^{14}$ flop/s $\approx$ 111 Tflop/s	$2 \cdot 10^{17}/1.8 \cdot 10^{3} = 1.11 \cdot 10^{14}$. "Circa 1 Pflop/s" va giustificato come margine ($R_{max} < R_{peak}$, $k > 1$)
Lez1 s.27	24 giorni	$\approx$ 23.1 giorni	$2 \cdot 10^{6}/86400 = 23.15$ (arrotondamento)
Lez7 s.22–23	$\alpha = 0.032$ costante; colonna $(1-\alpha)/p$; $E_{16} = 0.3$	$S_p = T_1/T_p$ della II strategia; $E_{16} \approx 0.39$	$T_{16}(32) = 5$, $S = 6.2$, $E = 6.2/16 = 0.3875$; con α fisso $S_{16} \approx 10.8$ (§3.1)

5. Come esporlo all'orale

DEFINIZIONI IN FORMA ACCADEMICA

Motivazione del calcolo parallelo Lez1 s.29–37

«Il calcolo parallelo è motivato dalla necessità di risolvere problemi su larga scala entro un tempo utile. Nel modello $\tau = k \cdot T(N) \cdot \mu$, la riduzione di μ incontra limiti tecnologici e la riduzione di $T(N)$ limiti teorici, legati all'esistenza di algoritmi ottimali. Si interviene allora distribuendo il carico computazionale su p unità processanti, sostituendo $T(N)$ con $T_p(N)$.»

T_1 e T_p Lez6 s.7, 13–14, 21 Lez7 s.10

«$T_1(N)$ è la complessità computazionale su un'unica unità processante; $T_p(N)$ è la complessità su p unità processanti, misurata in passi temporali. Poiché in parallelo più operazioni avvengono allo stesso passo, la complessità di tempo da sola non basta a valutare un algoritmo parallelo. Per convenzione T_1 è la complessità dell'algoritmo parallelo eseguito su un solo processore.»

Esempio di derivazione di T_p Lez6 s.31, 38

«Nella somma di N numeri con la II strategia ogni processore somma localmente N/p numeri in $N/p - 1$ passi; poi le p somme parziali vengono raccolte ad albero, dimezzandosi a ogni passo, in $\log_2 p$ passi. Quindi $T_p = N/p - 1 + \log_2 p$.»

Speed-up Lez6 s.26–27

«Lo speed-up $S_p = T_1/T_p$ misura la riduzione del tempo di esecuzione su p processori rispetto a un processore. Vale $S_p \leq p$; lo speed-up ideale $S_p = p$ è raggiunto dagli algoritmi full parallel.»

Overhead totale Lez6 s.42–43 Lez7 s.5

«L'overhead totale $O_h = pT_p - T_1$ misura quanto non si è riusciti a parallelizzare. Ricavando $T_p = (O_h + T_1)/p$ e sostituendo si ottiene $S_p = p/(O_h/T_1 + 1)$, quindi lo speed-up è ideale se e solo se $O_h = 0$. Per la somma con la II strategia $O_h = p \log_2 p - p + 1$, crescente in p.»

Efficienza Lez6 s.49–52

«L'efficienza $E_p = S_p/p$ misura quanto l'algoritmo sfrutta il parallelismo del calcolatore; vale $0 < E_p \leq 1$, con efficienza ideale pari a 1. Per la somma di 16 numeri p = 8 dà lo speed-up massimo, ma p = 2 dà l'efficienza massima: lo speed-up da solo non basta.»

Legge di Ware-Amdahl Lez7 s.13, 16, 20–21, 34

«Decomponendo $T_1 = T_s + T_c$ e assumendo che la parte concorrente si ripartisca tra i p core, $T_p = T_s + T_c/p$. Normalizzando $T_1 = 1$ e $T_s = \alpha$ si ottiene $S_p = 1/(\alpha + (1-\alpha)/p)$. Per $p \to \infty$ lo speed-up è limitato da $1/\alpha$: a dimensione fissata esiste un p_{max} oltre il quale le prestazioni degradano. A p fissato, al crescere di n, $\alpha \to 0$ e l'efficienza tende a quella ideale, entro il limite n_{max} imposto dalla memoria.»

Trade-off generale

«Aumentare p riduce il lavoro per processore ma aumenta l'overhead della collezione dei risultati; la scelta di p è un compromesso tra tempo di risposta ed efficienza nell'uso delle risorse.»

FORMULAZIONI DA EVITARE

- ✗ «Il parallelo serve ad andare più veloce» → ✓ «a ridurre il tempo di risposta di problemi su larga scala entro un tempo utile».
- ✗ «Con p processori vado p volte più veloce» → ✓ «$S_p \leq p$, con uguaglianza solo per algoritmi full parallel».
- ✗ «L'efficienza è lo speed-up» → ✓ «l'efficienza è lo speed-up normalizzato rispetto al numero di processori».
- ✗ «L'overhead è il tempo di comunicazione» → ✓ «l'overhead totale è $pT_p - T_1$, il lavoro aggiuntivo rispetto al sequenziale; nel nostro modello multicore deriva dalla collezione dei risultati».
- ✗ «$O_h = p \log_2 p - p$» (come nella slide) → ✓ «$p \log_2 p - p + 1$», da verificare al volo con N = 16, p = 2: $2 \cdot 8 - 15 = 1$.